

O que o Back-end precisa coletar — Entrega da Ciência de Dados

Hackathon ONE · Alura + Oracle (OCI) — G9 Team 20

Checklist objetivo: exatamente quais arquivos pegar, de onde, e o que fazer com cada um. Nenhuma ambiguidade sobre qual pasta ou qual versão usar.

⚠ Qual versão usar: a "Versão Final", não a original

O projeto tem duas versões no Drive. **Use sempre a Versão Final** — é a mais recente, mais limpa, e com os modelos já corrigidos (inclusive o erro de classificação do "Combustível", corrigido só na versão final).

HACKATHON G9 - TEAM 20/

└─ notebooks/, datasets/, modelos/ ← Versão 1 (NÃO usar — histórico)

└─ versao_revisada_final/

└─ modelos/ ←  USAR ESTA PASTA

Caminho completo no Drive:

/MyDrive/Hackathon_OCI_G9/HACKATHON G9 - TEAM 20/versao_revisada_final/modelos/

Os 5 arquivos obrigatórios (e só estes)

#	Arquivo	Para que serve
1	<code>vetorizador_tfidf.pkl</code>	Converte a descrição da transação (texto) em números

#	Arquivo	Para que serve
2	<code>modelo_categoria_producao.pkl</code>	Prevê a categoria da transação (Alimentação, Transporte, etc.)
3	<code>codificador_categorias.pkl</code>	Traduz o número devolvido pelo modelo de volta para o nome da categoria
4	<code>modelo_perfil_producao.pkl</code>	Prevê o perfil financeiro (Saudável / Em observação / Em risco)
5	<code>codificador_perfil.pkl</code>	Traduz o número devolvido pelo modelo de volta para o nome do perfil

Não são necessários (ficam de fora da entrega): `modelo_categoria_deep_learning.h5`, `tokenizador_categoria.pkl`, `modelo_perfil_deep_learning.h5`, `escalador_perfil.pkl` — são só comparação técnica documentada, não entram em produção.

O código do pipeline (a lógica que usa esses 5 arquivos)

Está pronto e testado no notebook `04_pipeline_integracao_final.ipynb` (pasta `versao_revisada_final/notebooks/`). As funções a copiar são:

`carregar_artefatos_producao()`

`classificar_categorias_transacoes()`

`calcular_resumo_gastos()`

`prever_perfil_financeiro()`

`gerar_recomendacoes()`

`analisar_financas()` ← função principal, chama todas as outras

Essas 6 funções, juntas, formam o "cérebro" que precisa rodar dentro da API — seja em Java (via OCI Function) ou em Python (direto na API).

Documentos de apoio (para decidir a arquitetura)

Documento	Quando usar
CONTRATO_INTEGRACAO_BACKEND.md	Se o time decidir manter Java
CONTRATO_INTEGRACAO_BACKEND_PYTHON.md	Se o time decidir migrar para Python
documentacao_tecnica.md	Para entender o "porquê" de qualquer decisão tomada (inclui o guia de decisão Java vs Python)

Formato de entrada que a API precisa aceitar

```
{  
  
  "renda_mensal": 4500,  
  
  "nivel_endividamento": 25,  
  
  "frequencia_poupanca": "Media",  
  
  "transacoes": [  
  
    { "descricao": "Supermercado", "valor": 420 },  
  
    { "descricao": "Combustivel", "valor": 300 },  
  
    { "descricao": "Streaming", "valor": 40 }  
  
  ]  
}
```

Formato de saída que a API precisa devolver

```
{  
  
  "perfil_financeiro": "Saudavel",  
  
  "probabilidade": 0.87,  
  
  "resumo_gastos": {  
  
    "Alimentacao": 420.0,  
  
    "Transporte": 300.0,  
  
    "Lazer": 40.0  
  
  },  
  
  "recomendacoes": [  
  
    "Manter o padrão atual de organização financeira",  
  
    "Considerar investir o excedente mensal para objetivos de longo prazo"  
  
  ]  
  
}
```

Passo a passo resumido para quem for integrar

1. Baixar os 5 arquivos `.pkl` da pasta `versao_revisada_final/modelos/`
2. Ler o `documentacao_tecnica.md` — seção "Guia de Decisão: Integração do Back-end" — e decidir Java ou Python com o grupo
3. Seguir o contrato correspondente à escolha (`CONTRATO_INTEGRACAO_BACKEND.md` ou `..._PYTHON.md`)
4. Copiar a lógica das 6 funções do pipeline (notebook `04_pipeline_integracao_final.ipynb`)
5. Testar localmente com o exemplo de entrada acima antes de subir para a OCI
6. Fazer upload dos 5 `.pkl` para o **OCI Object Storage**

7. Fazer deploy da aplicação (**OCI Compute** ou **OCI Functions**, conforme a escolha do passo 2)